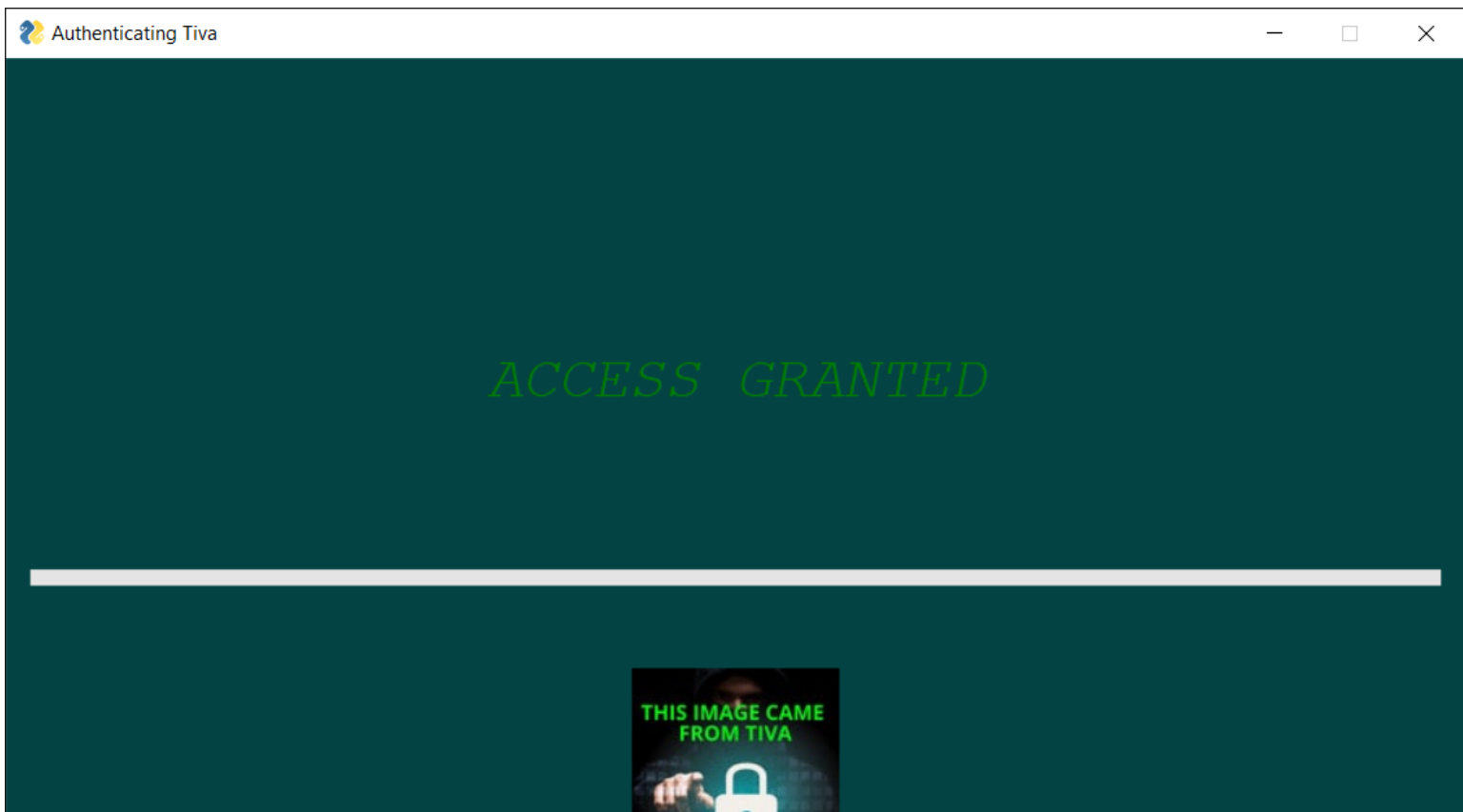


Embedded Zero-Knowledge Authentication

Project submitted by:

Yosi Schlakman [REDACTED]

Yehoshua Gruenspecht [REDACTED]



Computer Science, JCT, Fall semester of 2021-2022

Instructor: Prof. Arie Haenel

Table of Contents

Contents

Table of Contents.....	2
Abstract.....	3
Introduction	4
Authentication	4
The Problem With Simple Authentication	4
Zero-Knowledge Authentication.....	5
Feige-Fiat-Shamir Identification Scheme	6
Research and Work Distribution	8
Research.....	8
Work Distribution	8
Implementation	9
Overall.....	9
Security	9
Future Improvements	10
Conclusion.....	11
Acknowledgments.....	11
Appendix	12

Abstract

As we all know, authentication is a key component in software security. The most common way to authenticate a person or machine is via password that only the one being authenticated should know. However, if there were to be an eavesdropper, this password can easily be heard and anyone can now be falsely authenticated. We wish to solve this problem by authenticating without revealing the password. This is called “Zero-knowledge authentication”. In the following project we researched the topic of Zero-knowledge authentication and created an embedded system that implements the Feige-Fiat-Shamir method for Zero-knowledge authentication. We will prove that our system works as an authentication system and that it fulfils the criteria for being Zero-knowledge.

Introduction

Authentication

In software security, authentication is when we ask the question: “Are you who you say you are?” to be sure we are communicating with the right person or machine. Peggy can authenticate herself in a number of ways.

1. Something Peggy knows

This can for example be a password that only Peggy knows therefore if Peggy tells Victor the password, Victor will know for sure that he’s talking to Peggy. Of course it can’t be this simple as we will see later.

2. Something Peggy has such as a physical key or magnetic chip.

3. Something Peggy is. For example, biometrics, fingerprint, face ID and even handwriting.

Authentication can be man-to-machine or machine-to-machine. In our project we will be dealing with machine-to-machine authentication. We will connect a Tiva Launchpad to a PC via serial and run an authentication protocol where the Tiva Launchpad proves it is part of a group that can communicate with the PC by knowing something only a member of the group will know.

The Problem With Simple Authentication

As we already stated, Peggy can authenticate herself by proving she knows something only a member of the group with access would know. The simplest way to do so would be to simply tell said secret to Victor and Victor will know if it is correct or not.

This method is problematic, because if Eve were to eavesdrop while Peggy revealed her secret to Victor, or if Eve got a glimpse victors list of secrets (got access to the database) - Eve now knows Peggy’s secret and will be able to convince Victor that she is Peggy and therefore has all of Peggy’s capability’s that Eve may not have otherwise.¹

One possible solution is to use a One Time Password. In the case of an OTP eavesdropping is useless because the password will change next time it needs to be used. The problem with OTP is it isn’t easy and cheap for the client. There are other solutions such as the many challenge-response authentication protocols however, in these protocols, the password stays the same every time and can be deciphered and therefore still vulnerable. Another disadvantage is that all the encrypting and decrypting takes time.

¹ See appendix 1

Zero-Knowledge Authentication

A Zero-knowledge proof (ZKP) is a mathematical technique to verify the truth of information without revealing the information itself. There are many applications of ZKP such as:

- **Finance:** ING uses ZKPs that allow customers to prove that their secret number lies in a known range. For example, a mortgage applicant can prove that their income is in the admissible range without revealing their exact salary.
- **Online voting:** ZKPs can allow voters to vote anonymously and to verify that their vote was included in the final tally.
- **Authentication:** ZKPs can be used to authenticate users without exchanging secret information such as passwords.²

In the following project, we will use Zero-knowledge proof as an authentication system to solve the problem with simple authentication stated above.

Zero-knowledge authentication is where Peggy authenticates herself by proving to Victor that she knows a secret only she should know without revealing the secret to Victor. This can be done by Victor asking Peggy a series of questions, that by answering correctly consistently, prove that Peggy knows the secret. For example, the Two Balls and Color Blind Friend: Imagine your friend is red-green colour-blind (while you are not) and you have two balls: one red and one green, but otherwise identical. To your friend they seem completely identical and he is sceptical that they are actually distinguishable. You want to prove to him they are in fact differently-coloured, but nothing else; in particular, you do not want to reveal which one is the red and which is the green ball.

Here is the proof system. You give the two balls to your friend and he puts them behind his back. Next, he takes one of the balls and brings it out from behind his back and displays it. He then places it behind his back again and then chooses to reveal just one of the two balls, picking one of the two at random with equal probability. He will ask you, "Did I switch the ball?" This whole procedure is then repeated as often as necessary.³

A Zero-knowledge technique must satisfy three conditions:

1. Completeness: If this statement is true, the honest verifier will always be convinced of this fact by an honest claimant.
2. Soundness: If the statement provided by the claimant is false, no cheating claimant will be able to convince the honest verifier that it is actually true.
3. Zero-knowledge: If the statement is true, no cheating verifier can learn anything other than this property. This is proved by showing that every cheating verifier has some simulator that can give a fake transcript of interaction between the claimant and the verifier when it has to prove about the relationship.⁴

Let's consider the Two Ball example.

² Taken from <https://research.aimultiple.com/zero-knowledge-proofs/>

³ Taken from Wikipedia [ZK Two Balls](#)

⁴ Taken from <http://ethesis.nitrkl.ac.in/5755/1/110CS0371-2.pdf> page 19

Completeness: If the balls really are two different colors, the friend will be convinced when you tell him that he switched the balls (assuming you are honest).

Soundness: If someone who doesn't actually know if the balls were switched or not would try to convince the color blind friend that he does know, he would have a 50% chance of guessing correctly. Now let's assume that the process was repeated 10 times. That means that the cheater now has a $\frac{1}{2^{10}} = \frac{1}{1024} = 0.09\%$ chance of tricking the friend. Since the probability of tricking the verifier is so low, it's safe to say the no cheater will be able to convince the verifier that he is an honest prover.

Zero-knowledge: The friend is still color blind and since you never tell him which ball is which color, only that they are different colors, he still won't know.

Feige-Fiat-Shamir Identification Scheme

For our project we decided to use the Feige-Fiat-Shamir identification scheme (FFS) to implement our Zero-knowledge authentication system. Here's how it works:

1. Peggy choses two prime numbers p and q where $p \equiv q \equiv 3 \pmod{4}$
2. $N = p * q$
3. $S_1, S_2, \dots, S_k$ are co-prime to N and are Peggy's secrets.
4. $V_1, V_2, \dots, V_k$ are sent to Victor while $\rightarrow V_i = S_i^2 \pmod{N}$.
5. Peggy chooses random $r \in \mathbb{R}$ and $s \in \{1, -1\}$ and calculates $x \equiv s * r^2 \pmod{N}$
6. Peggy sends x to Victor.
7. Victor chooses $a_1 a_2 \dots a_k \mid a_i \in \{0, 1\}$ and sends to Peggy
8. Peggy calculates $y \equiv r * (S_1^{a_1} S_2^{a_2} \dots S_k^{a_k}) \pmod{N}$ and sends y to Victor
9. Victor verifies that $y^2 \equiv \pm x(V_1^{a_1} V_2^{a_2} \dots V_k^{a_k}) \pmod{N}$
10. Run multiple times until Victor is satisfied.⁵

Now let's check that FFS satisfies our 3 conditions for Zero-knowledge proof.

Completeness: If Peggy really knows the secrets, Victor will know after receiving both the x and the y values because $y^2 \equiv r^2((S_1^{a_1})^2(S_2^{a_2})^2 \dots (S_k^{a_k})^2) \pmod{N} = r^2(S_1^{2a_1} S_2^{2a_2} \dots S_k^{2a_k}) \pmod{N} = r^2((S_1^2)^{a_1} (S_2^2)^{a_2} \dots (S_k^2)^{a_k}) \pmod{N} = r^2(V_1^{a_1} V_2^{a_2} \dots V_k^{a_k}) \pmod{N} = x(V_1^{a_1} V_2^{a_2} \dots V_k^{a_k}) \pmod{N}$

Only if $\forall i \rightarrow V_i = S_i^2 \pmod{N}$ meaning, Peggy knows every S_i value. Therefore, Victor will be convinced by a true statement.

Soundness: Eve will be able to learn the V_i values by eavesdropping. However, this will not be enough to be authenticated. Eve will now still have to guess all of Victor's a_i values and then pick a random y to calculate $x \equiv y^2 v_1^{-a_1} v_2^{-a_2} \dots v_k^{-a_k} \pmod{n}$ and then send x to Victor. Now when Victor sends $a_1 a_2 \dots a_k$ Eve will simply return y and because of the x Eve sent Victor:

⁵ See appendix 2

$$y^2 \equiv x v_1^{a_1} v_2^{a_2} \dots v_k^{a_k} \pmod{N} =$$

$$y^2 (v_1^{-a_1} v_2^{-a_2} \dots v_k^{-a_k}) \pmod{N} (v_1^{a_1} v_2^{a_2} \dots v_k^{a_k}) \pmod{N} \equiv y^2$$

Victor will be falsely satisfied!

However, FFS is still sound because the probability of Eve guessing all a_i values correctly is $\frac{1}{2^k}$ and the probability of Eve guessing correctly for all t rounds of the protocol is $\frac{1}{2^{kt}}$. For example, if we choose $k = 10$ and $t = 5$ we will get $\frac{1}{2^{50}} = \frac{1}{1.125 * 10^{12}} = 88 * 10^{-14} = 0.88 * 10^{-14} \% \Rightarrow 0$

In other words, the probability of Eve identifying as Peggy is zero (extremely low).

Zero-knowledge: Even if Eve learns the V_i values, she still won't be able to know the secrets because it's extremely hard to determine a modular square root when the modulus' factorization is unknown (like the discrete log problem).

Research and Work Distribution

Research

We started the research process by reading Wikipedia articles and different papers written on the topic such as the ones quoted in the report.

At first, we tried creating our own Zero-knowledge authentication protocol using modulus calculations and even wrote it down in our initial proposal. Thankfully our instructor, Prof. Arie Haenel, read our proposal and explained to us all the vulnerabilities with our protocol. He also opened our eyes to already existing Zero-knowledge protocols and explained it would be better to use an already existing protocol that has been proven to work instead of working hard to come up with our own protocol that won't work as well

That's when we thanked him for his help and went on to research different Zero-knowledge protocols such as the Feige–Fiat–Shamir identification scheme, Okamoto-Schnorr, Guillou-Quisquater and of course the original Fiat-Shamir. In the end, we decided to go with FFS because it's simple, commonly used and well known (which means that a lot of papers were written on the topic which is good for research purposes).

Now all we had to do was learn the necessary coding skills needed for writing the software and creating a connection between our Tiva Launchpad and PC using Energia. We also wanted to create a GUI that can show how the whole protocol works and in order to do that we had to learn a bit about the Python library PySimpleGUI because it seemed like a simple way to create an efficient GUI (it has "simple" in the name!).

Work Distribution

Once all of the research was done, it was time to get to work. We ordered the work needed to be done like this:

- Create a program that will establish a connection between the Tiva and the PC.
- Write the actual source code for the PC to run the FFS protocol.
- Write code for a GUI and merge that with the already existing code.
- Create a slideshow presentation. This was considered another step because there wasn't really a lot of software involved in this project, so we wanted to make sure that we did a good job explaining the general idea of ZKP before presenting our project.
- Write this report.

Yosi worked on the first two steps while Yehoshua worked on the third and fourth steps.

Once Yosi finished with the first two steps, Yehoshua started working on this report and Yosi worked on merging the GUI with the rest of the code to have the work well together. It was during this time that Yosi also worked on some security measures.

Implementation

Overall

To implement a Zero-knowledge authentication system we used an embedded device (Tiva Launchpad) as Peggy trying to authenticate itself to Victor our PC.

The order of events goes like this:

0. Run the program.
1. Connect the Tiva to the PC using the USB cable.
2. The PC will now initiate a two-way handshake with the Tiva.
3. Now that the initial connection has been made, we can run the FFS identification scheme to authenticate the Tiva while all of the communication is being done through serial.
4. Run the FFS identification process t times. We chose $t = 5$ because we decided it was enough to make the probability of an eavesdropper guessing extremely low but not to many times that the whole process feels long and tedious for the user.

We also created a GUI to show the user every step that is going on the software level and the progress being made.

Our program fulfils the criteria for Zero-knowledge proof for the same reasons that FFS does.

Security

- We chose to use a secret vector of length $k = 4$, and to run the FFS identification scheme $t = 5$ times. The reason we chose these numbers are as follows:
 - We experimented with various t values and found that $t = 5$ is the “sweet spot” for the identification (in our program) to not take too long, but also not be so short that the user can’t see what’s going on.
 - Once we found our t value, we knew we wanted the probability of successful cheating to be over one in a million, so we chose $k = 4$ - giving a cheater a probability of $2^{-kt} = 2^{-4*5} = \frac{1}{1,048,576}$ for success, but not increasing the probability (and therefore the execution time) more than necessary.

Of course if our program was to be modified and used for applications that require (almost) instantaneous authentication – such as smart card authentication, the t and k values would have to be reconsidered for dealing with the time constraints and possibly lowering the probability of malicious success if the application allows it (since a probability of one in a million is not needed for many application – such as a simple key card access program).

- We chose to use a prime number of 2048-bit length for N in the FFS scheme. The reason for our choice is that we found (via online research) this was the recommended length for RSA primes, in order to prevent factorization of N using modern technology and near

future technology. Additionally, we found that all the official “RSA numbers”⁶ of 2048-bit length have not yet been factorized, which gave us confidence in our choice.

- While our program isn’t vulnerable to Eve eavesdropping on the communications, it is vulnerable to man in the middle attacks in which Eve would not just be a listener, but rather she would be in the middle of Peggy and Victor’s communication. In this way, Eve could relay all of Peggy’s communications to Victor and all of Victor’s communications to Peggy – causing Victor to verify Eve when he thinks he’s verifying Peggy. While this is of concern, this still doesn’t allow Eve to know Peggy’s secret, only to “prove” to Victor that she does. Additionally, this wouldn’t allow Eve to authenticate whenever she wants – she can only authenticate when Peggy is actively trying to connect.
- After the authentication, it is assumed that the device connected to the PC serial port is the Tiva. This allows for a theoretical attack in which right after the Tiva authenticates, an attacker connects to the serial port and essentially “hijacks” the Tiva’s session. We tested for this attack, and the serial connection used automatically closes when the Tiva is disconnected and doesn’t allow a newly connected device to use the same port. However, there probably is a way to exploit this (that we don’t know of) and therefore this is a vulnerability.

Future Improvements

- As mentioned in the security section, our program is vulnerable to the man-in-the-middle attack. This can be improved upon by adding some symmetric key encryption scheme to the communication between the PC and the Tiva (such as RSA). If the scheme is too slow, it can be used to initially transfer shared keys for symmetric-key cryptography which is faster.
- Also mentioned in the security section, is the “session hijacking” problem. This can be improved upon by implementing the previous point and implementing sessions (similar to the ones used in most browsers) – in which the Tiva gets a session key (over an encrypted line) that he gives the PC when he wants to communicate with it. This way the PC knows that it is really the Tiva that is connected and that the session hasn’t been hijacked.
- We chose to load an image from the Tiva to our program on the PC in order to show that the Tiva is now trusted. However, more elaborate and abundant functionality can be thought of and implemented. We chose not to focus too much on this area of the project, since the requirement was to make an authentication system – but it could make the program more practical and elevate its “cool” factor.

⁶ https://en.wikipedia.org/wiki/RSA_numbers

Conclusion

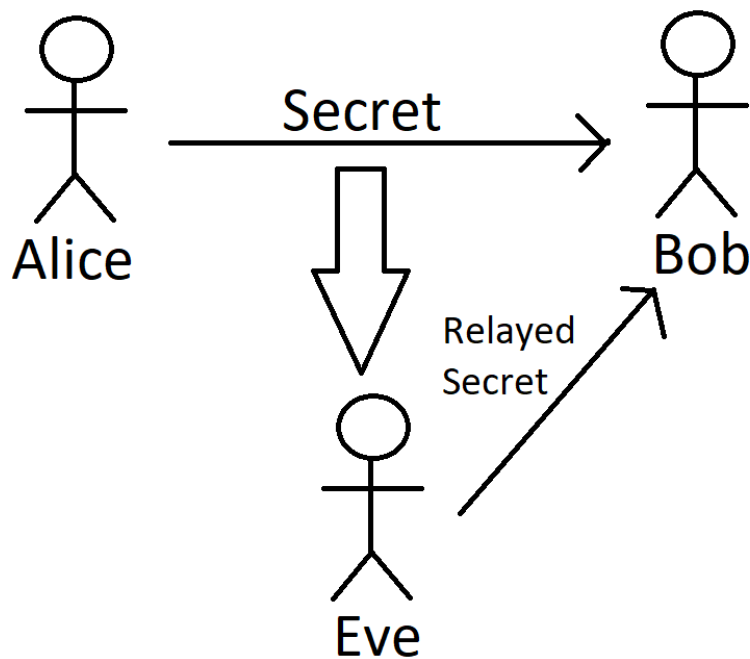
- We described an embedded Zero-knowledge authentication system using a Tiva Launchpad and the Feige-Fiat-Shamir identification scheme.
- It's more secure than regular passwords.
- It's easier to use than OTP.
- It's quicker and more secure than a Challenge-response technique or an encrypted password.
- Just like everything else in software security, there is still room for improvement.

Acknowledgments

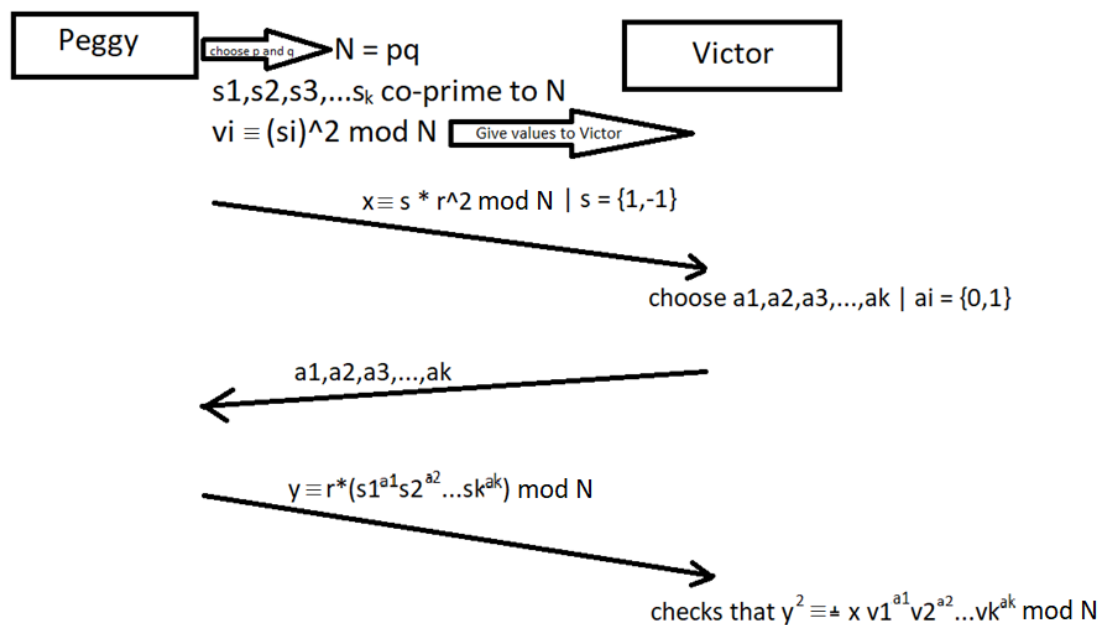
We would like to thank Prof. Arie Haenel for his helpful remarks and for guiding us through this past semester and this project in particular.

Appendix

1.



2.



Repeat the process with different r and a_i values until Victor is satisfied

